

Elements of AIML

Applications of AI and Machine Learning

For Batch: B49-50, B71-72

How Do We Apply AI/ML in Real Life?

- ✓ Identifying real problem
- ✓ Understanding domain
- ✓ Choosing correct ML approach
- ✓ Building complete solution pipeline

How Do We Apply AI/ML in Real Life?

Step 1 - Problem Identification: What problem are we trying to solve?

Examples:

Domain	Problem
Banking	Loan default prediction
Healthcare	Disease diagnosis
Education	Student performance prediction
Retail	Customer churn prediction
Insurance	Fraud detection

How Do We Apply AI/ML in Real Life?

Step 2 - Problem Type Identification

Classify the ML problem:

Problem Type	Example
Classification	Fraud / No Fraud
Regression	House price prediction
Clustering	Customer segmentation
Recommendation	Product recommendation
NLP	Chatbot, sentiment analysis

How Do We Apply AI/ML in Real Life?

Step 3 - Dataset Understanding

Questions:

- What features are available?
- What is target variable?
- Are labels available?
- Missing values?
- Class imbalance?

Example: Loan dataset

- Income
- Credit score
- Loan amount
- Employment status
- Default (Target)

How Do We Apply AI/ML in Real Life?

Step 4 - ML Pipeline

Complete Machine Learning Pipeline:

1. Problem Definition
2. Data Collection
3. Data Preprocessing
4. Exploratory Data Analysis
5. Feature Engineering
6. Model Selection
7. Model Building
8. Evaluation
9. Deployment

Example Case Study: Banking Domain

Problem: Loan Default Prediction

Goal: Predict whether customer will default loan or not

Type: Classification Problem

Dataset Example:

Income	Credit Score	Loan Amount	Employment	Default
50000	700	200000	Salaried	No
30000	550	300000	Self-employed	Yes

Step-by-Step Python Workflow

PART A — Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix
```


Step-by-Step Python Workflow

Load Dataset

```
df = pd.read_csv("loan_data.csv")  
  
print(df.head())  
print(df.info())  
print(df.isnull().sum())
```

Note:

- ✓ Understand data first
- ✓ Never start directly with model

Step-by-Step Python Workflow

Handle Missing Values

```
df.fillna(df.mean(numeric_only=True), inplace=True)
```

Encode Categorical Features

```
le = LabelEncoder()  
df['Employment'] = le.fit_transform(df['Employment'])  
df['Default'] = le.fit_transform(df['Default'])
```

Note: ML models need numerical input

Step-by-Step Python Workflow

Feature Selection

```
X = df[['Income', 'Credit Score', 'Loan Amount', 'Employment']]  
y = df['Default']
```

X = Input

y = Target

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Training → learning

Testing → evaluation

Step-by-Step Python Workflow

Model Training

```
model = LogisticRegression()  
model.fit(X_train, y_train)
```

Since target is Yes/No → Classification

Prediction

```
y_pred = model.predict(X_test)
```

Evaluation

```
print("Accuracy:", accuracy_score(y_test, y_pred))  
print(confusion_matrix(y_test, y_pred))
```

Step-by-Step Python Workflow

Interpretation

- Why did model fail on some cases?
- Can recall be more important than accuracy?

Same Framework Across Domains

Same pipeline applies to:

Healthcare → Disease prediction

Education → Dropout prediction

Retail → Customer churn

Insurance → Fraud detection

Environment → Pollution prediction

AI for Society, Women & Environment

Examples:

- Women safety prediction systems
- AI for domestic violence detection
- Air pollution monitoring
- Flood prediction
- Waste management
- Smart agriculture

Case Study: Air Pollution Monitoring using Machine Learning

Problem Statement:

- Government agencies want to predict Future Air Quality Index (AQI) using pollution and weather-related features.

This helps in:

- Public health warnings
- Pollution control planning
- Smart city management
- Environmental monitoring

Problem Type

Regression Problem

because:

AQI = Continuous Numerical Value

Example:

AQI = 85, 140, 230

Sometimes it can also be:

Sometimes it can also be: **Classification**

if we predict:

- Good
- Moderate
- Poor
- Severe

Dataset Used

Source: <https://www.kaggle.com/datasets/waqi786/global-air-quality-dataset>

Sample Dataset Table:

PM2.5	PM10	NO2	Temp	Humidity	AQI
120	180	45	32	65	210
55	90	20	28	70	95

Full Machine Learning Pipeline: Problem → Data → Preprocessing → EDA → Model → Evaluation → Deployment

PART 1: DATA PREPROCESSING

Why Data Preprocessing?

Real-world environmental data is messy:

Problems include:

- Missing values
- Outliers
- Duplicate rows
- Wrong units
- Categorical values
- Different scales

If preprocessing is poor: Model performance becomes poor

Step 1 - Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

Step 2 - Load Dataset

```
df = pd.read_csv("air_quality.csv")
```

```
print(df.head())
```

```
print(df.info())
```

```
print(df.shape)
```

```
print(df.isnull().sum())
```

Step 3 - Missing Value Handling

Example:

Sensor failure → missing values

Fill with mean value:

```
df.fillna(df.mean(numeric_only=True), inplace=True)
```

Alternative Missing Value Methods:

Method	Use Case
Mean	Normal numeric data
Median	Outlier-heavy data
Mode	Categorical data
Drop rows	Very few missing values

Step 4 - Duplicate Removal

```
df.drop_duplicates(inplace=True)
```

Step 5 - Outlier Detection

Example:

PM2.5 = 9999

- This is unrealistic
- Must be handled

Outlier Detection using Boxplot

```
plt.boxplot(df['PM2.5'])  
plt.title("Outlier Detection for PM2.5")  
plt.show()
```

Outlier Removal using IQR

```
Q1 = df['PM2.5'].quantile(0.25)  
Q3 = df['PM2.5'].quantile(0.75)  
  
IQR = Q3 - Q1  
  
lower = Q1 - 1.5 * IQR  
upper = Q3 + 1.5 * IQR  
  
df = df[(df['PM2.5'] >= lower) & (df['PM2.5'] <= upper)]
```


Step 6 - Feature Scaling

Why needed?

Because:

Feature	Range
Temperature	20–40
PM2.5	10–500

Large values dominate model

Standardization

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
features = ['PM2.5', 'PM10', 'NO2', 'Temp', 'Humidity']  
df[features] = scaler.fit_transform(df[features])
```

PART 2: EDA (Exploratory Data Analysis)

Why EDA?

Before modeling:

We must understand:

- feature relationships
- trends
- correlations
- anomalies

Distribution Plot

```
plt.hist(df['AQI'])  
plt.title("AQI Distribution")  
plt.show()
```

Scatter Plot

```
plt.scatter(df['PM2.5'], df['AQI'])  
plt.xlabel("PM2.5")  
plt.ylabel("AQI")  
plt.title("PM2.5 vs AQI")  
plt.show()
```

Correlation Matrix

```
import seaborn as sns  
sns.heatmap(df.corr(), annot=True)  
plt.title("Feature Correlation")  
plt.show()
```

PART 3: MODEL BUILDING

Feature Selection:

```
X = df[['PM2.5', 'PM10', 'NO2', 'Temp', 'Humidity']]
```

```
y = df['AQI']
```

Train-Test Split:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

PART 3: MODEL BUILDING

Train Model:

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

Prediction:

```
y_pred = model.predict(X_test)
```

PART 4: MODEL EVALUATION

Evaluation Metrics:

```
mae = mean_absolute_error(y_test, y_pred)
```

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

```
print("MAE:", mae)
```

```
print("RMSE:", rmse)
```

Interpretation:

Lower:

✓ MAE

✓ RMSE

means:

Better prediction performance